monara

01

SUBMISSION PACKAGE / 01 OF 03

Technical Documentation

System architecture, core algorithms and security design

Architecture → Algorithms → Security

COMPETITION

2026 Shenzhen International FinTech Competition (FinTechathon)

TRACK AND TOPIC

International Track, Topic A: Personal Finance Assistant

TEAM

Team 808

Contents

1 Purpose and assessment criteria 2
1.1 Product scope 3
1.2 Scope boundaries 3
2 System architecture and interface 3
2.1 Architecture overview 3
2.2 Runtime responsibilities 4
2.3 Primary data flows 4
2.4 Interface and PWA behavior 5
2.5 Reliability and recovery 5
2.6 Key architecture decisions 6
3 Core algorithms 6
3.1 Gmail extraction and classification 6
3.2 WhatsApp receipt extraction 7
3.3 Agent action model 7
3.4 Financial insights 8
3.5 Spending forecast 8
3.6 Reminder algorithm 8
3.7 Algorithm decisions and limitations 9
4 Security design 9
4.1 Trust model 9
4.2 Identity and authorization 9
4.3 Database ownership and integrity 10
4.4 Provider and webhook boundaries 10
4.5 Model safety 10
4.6 Browser and privacy controls 10
4.7 Destructive actions and permanent deletion 11
4.8 Security control matrix 11
4.9 Residual risks 11
5 Verification and readiness 12
5.1 Reproducible command gates 12
5.2 Browser acceptance 12
5.3 Controlled end-to-end journeys 12
5.4 Capability status 13

1 PURPOSE AND ASSESSMENT CRITERIA

Purpose and assessment criteria

Monara is a mobile-first personal-finance PWA that turns bank and e-wallet notifications into an owned financial ledger. It combines Gmail capture, human review, budgets, forecasts, savings goals, reminders, in-app chat and a verified WhatsApp channel without allowing a model or external provider to become the authority for identity or financial state.

This document is organized around the three technical criteria required for the submission. It explains the deployed architecture, the algorithms that transform financial evidence into useful outputs, and the security boundaries that protect each User.

Assessment criterion	Coverage in this document	Evaluation focus
System architecture	Browser PWA, authenticated backend, Agent Worker, database, queues, schedules and external providers	Clear runtime boundaries, reliable data flow and mobile usability
Core algorithms	Gmail extraction, typed classification, review confidence, deduplication, Agent permissions, forecasts, insights and reminders	Deterministic rules around model assistance and reproducible outputs
Security design	Identity, ownership, RLS, signed webhooks, same-origin writes, private caching rules, destructive confirmation and deletion	Untrusted input never becomes authority and cross-User access fails closed

1.1 Product scope

The financial ledger uses three transaction types: expense, income and exclusion. Exclusions keep reimbursements, self-transfers and other non-budget activity visible without counting them as spending or income. Transactions are either pending review or confirmed. Only confirmed expenses and income affect totals, budgets and insights.

The product supports:

- Supabase Google sign-in, separate Gmail consent and verified WhatsApp linking;
- an owned ledger with up to three Gmail connections, institution discovery, exact Custom Bank sender lists and human review;
- weekly and monthly Budgets, Reports, deterministic insights and separate spending forecasts;
- manually tracked Goals with private images and pace guidance derived from confirmed history;
- one permission-bounded Agent for web Chat and WhatsApp, including receipt-photo import;
- in-app Budget reminders, optional WhatsApp delivery and an installable PWA with a public-only offline fallback.

1.2 Scope boundaries

Monara records and analyzes financial evidence; it does not move money, initiate bank transfers or store payment-card credentials. Gmail access is read-only for selected institution senders. A prediction never becomes a confirmed transaction. Provider acceptance, delivery receipts and model accuracy require separate evidence from the deterministic controls described here.

The following capabilities remain Planned: voice-note transcription, persisted bill-due generation, generalized User timezones and full real-provider accuracy measurement. They are not counted as implemented behavior.

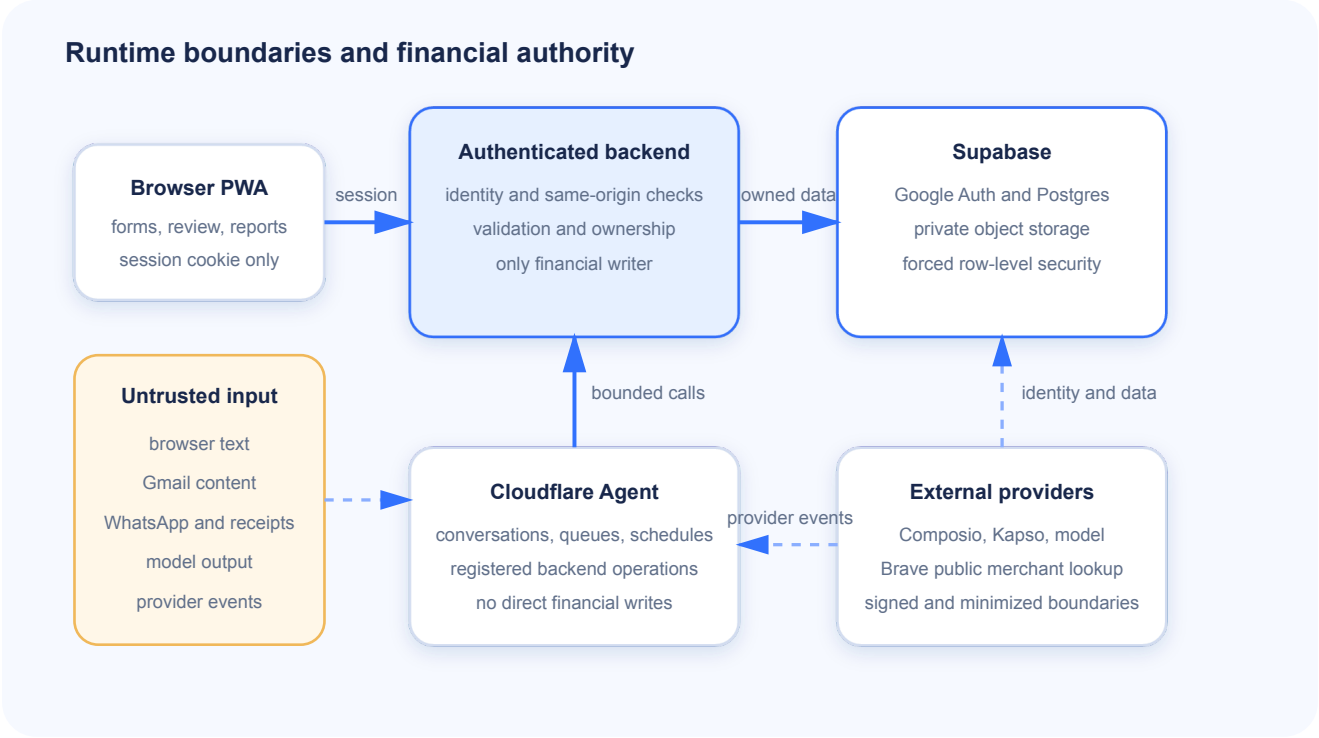
2 SYSTEM ARCHITECTURE AND INTERFACE

System architecture and interface

2.1 Architecture overview

Monara separates the interactive web experience from long-running Agent work. The Next.js application on Vercel owns authenticated APIs and all financial writes. A Cloudflare Worker runs conversations, queue consumers, scheduled work and provider-facing orchestration. Supabase supplies Google Auth, Postgres, private object storage and row-level security.

Composio manages Gmail authorization and read operations, Kapso transports WhatsApp messages, a model provider assists extraction and reasoning, and Brave Search may receive only a canonical public merchant name. Provider credentials remain on trusted server runtimes.



2.2 Runtime responsibilities

Component	Primary responsibility	Authority boundary
Browser PWA	Navigation, forms, review, reports, budgets, goals and Chat	Holds no provider credentials and cannot assert a User identity to backend services
Authenticated web backend	Session verification, validation, ownership checks and financial writes	Only runtime allowed to mutate the financial ledger
Agent Worker	Conversation loop, Gmail jobs, receipt interpretation, schedules and provider orchestration	Calls registered backend operations; never writes financial tables directly
Supabase	Identity, relational data, private images, constraints and RLS	Enforces owned rows and relationships independently of application filters
Composio	Gmail consent, selected-message retrieval and new-mail triggers	Separate from Google sign-in; mailbox identity is stored as a connection
Kapso	Signed WhatsApp transport	A phone number becomes usable only after explicit signed-in linking and verification
Model provider	Structured extraction, bounded classification and conversational reasoning	Receives minimized context; output is validated and never establishes authority

2.3 Primary data flows

Gmail to ledger

1. A signed-in User authorizes Gmail separately from application sign-in.

2. The backend discovers supported institution senders and maps each institution to an owned Financial Account.
3. Composio sends a signed new-mail event, or the daily catch-up requests the same import flow.
4. The Worker reads only mail matching exact configured senders and claims each message before extraction.
5. Structured extraction returns stated facts; deterministic validation rejects unsupported amounts, dates and mappings.
6. Each item is stored pending, categorized in a separate job and either confirmed by policy or left for review.
7. Stable provider and item identifiers make retries idempotent.

Chat or WhatsApp to action

The web Chat bridge verifies the browser session and origin before adding the trusted User context. WhatsApp verifies the provider signature, then resolves the sender through an explicitly verified link. Both channels enter the same Agent loop. The Agent can call only registered tools; the backend validates every argument, relationship, version and ownership rule before a change is accepted.

Derived data and scheduled work

Confirmed ledger changes update Budget usage and may create threshold reminders. Forecasts and insights are derived from confirmed records and remain distinct from financial facts. Four schedules coordinate cleanup, reminders, Gmail recovery and forecasts:

Schedule	Purpose
Every minute	Continue retryable account-deletion cleanup
Every 15 minutes	Dispatch eligible reminder deliveries
Daily at 01:00 UTC	Catch up Gmail messages missed by triggers
Daily at 17:00 UTC	Refresh forecasts at midnight in Jakarta

2.4 Interface and PWA behavior

The interface is English, light-only and designed first for a 390 x 844 mobile viewport. Protected screens display owned server data. The public demo uses labelled synthetic records and memory-only changes. A failed live load shows recovery rather than substituting sample data or partial totals.

History presents one ledger with All, Forecast, Expenses, Income, Exclusions and To review filters. Pending records receive a compact action state; confidence scores stay hidden. Exclusions are visually muted. Unknown manual or receipt dates remain visibly unresolved until supplied by the User.

Budgets show weekly and monthly groups with their own periods and limits. Reports provide monthly and yearly totals. Goals combine manual progress, a private image and a pace estimate derived from confirmed history without moving or reserving money. Chat history is stored server-side; temporary drafts and recovery receipts remain memory-only.

The PWA caches only explicitly public assets and a generic offline page. Financial pages, authentication responses, Chat and API data require connectivity and are never treated as offline assets.

2.5 Reliability and recovery

- Financial writes are serialized so an uncertain result is reconciled before another write begins.
- Request identifiers and record versions prevent accidental duplicate mutations.
- Undo restores the exact prior snapshot once within ten minutes and fails after a later edit.
- Gmail messages and extracted items have unique identities across retries.
- Chat history can retry a failed save for 24 hours without invoking the Agent or repeating a financial action.
- Account deletion separates immediate access revocation from retryable provider and storage cleanup.

2.6 Key architecture decisions

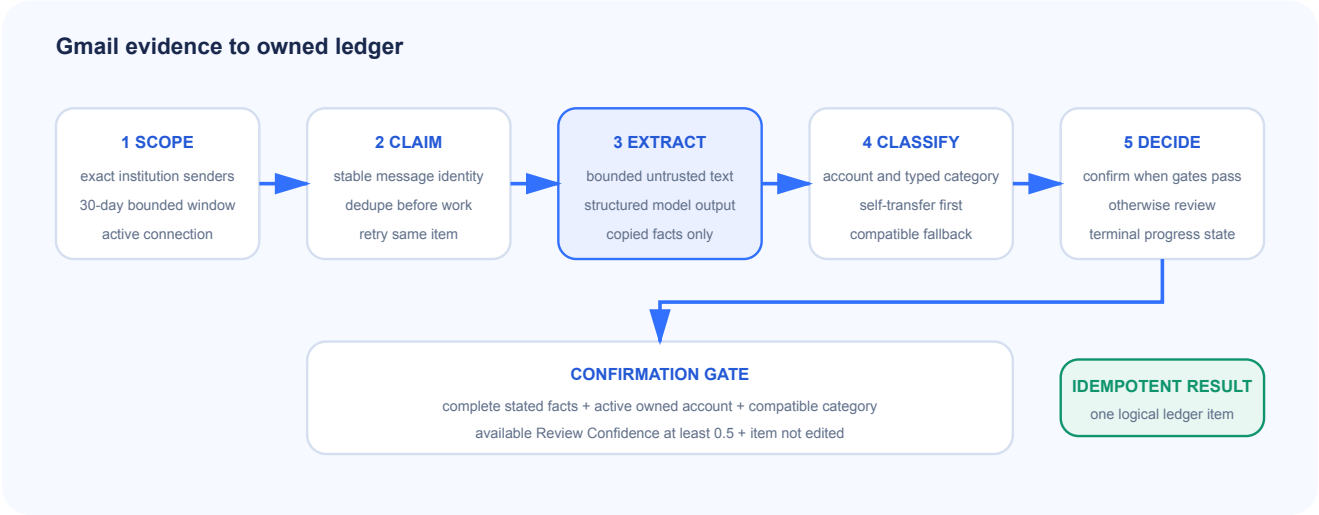
- **Backend-owned writes.** Direct Worker or browser database mutation was rejected because identity, validation, ownership and audit behavior must remain centralized.
- **Two deployable runtimes.** A single request runtime was rejected because queues, schedules and per-User WhatsApp state require durable background execution.
- **Separate consent per channel.** Google sign-in does not imply Gmail access, and a WhatsApp sender is not trusted until the signed-in User verifies the link.
- **Typed ledger semantics.** A separate transfer table was rejected. Exclusions preserve evidence and deduplication while remaining neutral to totals.
- **Public-only offline mode.** Caching personalized pages was rejected because stale financial and authentication state must never be exposed offline.

3 CORE ALGORITHMS

Core algorithms

3.1 Gmail extraction and classification

The import pipeline combines deterministic scope and validation with model-assisted extraction. The model may interpret a message, but code decides which messages are permitted, which facts are admissible and whether the result may affect the ledger.



Retrieval and normalization

Initial import covers the previous 30 days. Later syncs overlap the last cursor by one day, bounded by the same 30-day window, so delayed events can be recovered without an unbounded mailbox scan. Only exact senders registered for the User's active accounts are requested.

HTML, boilerplate and repeated content are removed before model use. The normalized body is capped at 4,000 characters, and one message can produce at most 20 candidate items. Whole IDR amounts and locations must appear in the source text. Implausible dates are rejected. If an email contains no usable stated date, its arrival day in Jakarta is used as a source-derived fallback; manual and receipt transactions keep the date unknown.

Account mapping and typed categories

The institution sender identifies a bank, not necessarily an account. A unique active account may be selected directly; otherwise the pipeline requires an unambiguous last-four or account-name match. The algorithm never invents a mapping.

Categories have immutable kinds matching expense, income or exclusion. Deterministic self-transfer matching runs first using the confirmed profile name and original counterparty evidence. Another personal name maps to Person; business purchases use a compatible specific category when available, then a typed Misc fallback. Only a canonical merchant from the bundled public catalog may be sent to Brave Search.

Review confidence and confirmation

The Review Confidence is the lower value when both extraction and classification confidence exist, or the sole available value when only one exists. Both missing, below 0.5, incomplete financial facts, an inactive account or an incompatible Category keeps the item pending. A User edit stops automatic application to that item.

Expense, income and exclusion can qualify for automatic confirmation when every policy gate passes. A confirmed exclusion remains neutral to financial totals. An exhausted classification retry ends in `needs_review`, so a reviewable item does not leave import progress running forever.

Deduplication

Two stable uniqueness boundaries are used:

- connection plus provider message identifier prevents one email from being processed twice;
- processed message plus item index prevents the same extracted line from creating two ledger rows.

Retrying queue work therefore resumes the same logical item instead of creating a new transaction.

3.2 WhatsApp receipt extraction

A verified linked sender may submit JPG, PNG or WebP receipt media. Bytes stay in memory during interpretation. A readable receipt creates exactly one pending transaction keyed by the WhatsApp message identifier. An unreadable amount creates nothing. Account matching must be unique, Category kind must match the extracted type, and a missing date stays unknown for review.

Receipt confidence never bypasses User review. Voice input remains Planned and receives an explicit unsupported response.

3.3 Agent action model

One Agent loop serves both web Chat and WhatsApp. It validates every tool call, returns tool results to the model, and stops after at most eight model steps. Several tool calls may appear in one step, so the limit bounds reasoning rather than financial operations.

Permission tier	Capabilities	User control
T0 - Read and derive	Summaries, lists, insights, Budget suggestions and forecasts	Executes immediately; no ledger mutation
T1 - Reversible write	Add or edit supported records, confirm or exclude pending items, create or edit Budgets and Undo	Validated ownership, version checks and durable action receipt
T2 - Destructive	Delete a transaction or Budget	Requires a bound confirmation that expires after five minutes
T3 - Not available	Credentials, provider connections, export and permanent User deletion	No conversational tool exists

Destructive confirmation is handled outside the model. The stored request binds the action, resource, version and expiry. Only the next message equal to `yes`, optionally followed by `.` or `!`, executes it. Ownership, version, expiry and prior consumption are checked again before deletion.

3.4 Financial insights

Insights use confirmed records only:

- **Savings rate:** confirmed income minus confirmed expenses, divided by confirmed income when income is positive.
- **Safe to spend:** remaining Budget allowance divided by the inclusive days left in its active period.
- **Overspend forecast:** confirmed spending pace projected to the end of the active period.
- **Unusual transaction:** at least three same-category expenses from the previous three complete months establish a median; a new expense at least twice that median is flagged.
- **Budget suggestion:** the previous three complete months form a proposed limit, which remains a draft until the User saves it.

Whole IDR amounts use integer arithmetic in the ledger. Missing data stays missing instead of being estimated as a financial fact.

3.5 Spending forecast

The forecast examines confirmed expenses from the previous 130 days and creates candidates through rules before the model is involved.

Monthly bills require activity in at least two different months, dates within two days of one another and a latest payment within 45 days. **Weekly habits** require the same weekday in at least four of the previous eight weeks. A merchant that matches both is treated as a habit after six or more occurrences. Amount is the median of recent payments, with the observed minimum and maximum retained as the range.

Rule confidence combines date consistency, amount stability and history depth. The model sees only candidate merchant names, dates and amounts. It cannot add an item or change an amount, and it may move confidence down by at most 0.25 or up by at most 0.15. Candidates below 0.6 are dropped and counted as uncertain. If the model is unavailable, the deterministic confidence is used.

One forecast atomically replaces the prior run. Predictions remain separate from transactions and never count as spending. A prediction becomes Paid only when a confirmed same-merchant expense occurs after forecast creation, within three days and within 35 percent of the expected amount.

3.6 Reminder algorithm

Confirmed expenses reevaluate the relevant weekly or monthly Budget. Crossing its warning percentage or 100 percent creates an in-app reminder with a durable key formed from Budget, period, limit and threshold. Re-evaluating the same condition cannot create a duplicate.

Optional WhatsApp delivery applies the following controls:

Control	Behavior
Quiet hours	Queue between 22:00 and 07:00 Jakarta time
Daily cap	Maximum three sent or in-flight claims per User day
Claim before send	One delivery row is reserved before the provider call
Recovery	A pre-provider claim may be reclaimed after five minutes
Unknown result	A stale sending result is not retried automatically
Batch size	Up to fifty eligible rows are processed outside quiet hours

In-app storage is authoritative for the reminder. Provider failure cannot undo the financial write or hide the warning.

3.7 Algorithm decisions and limitations

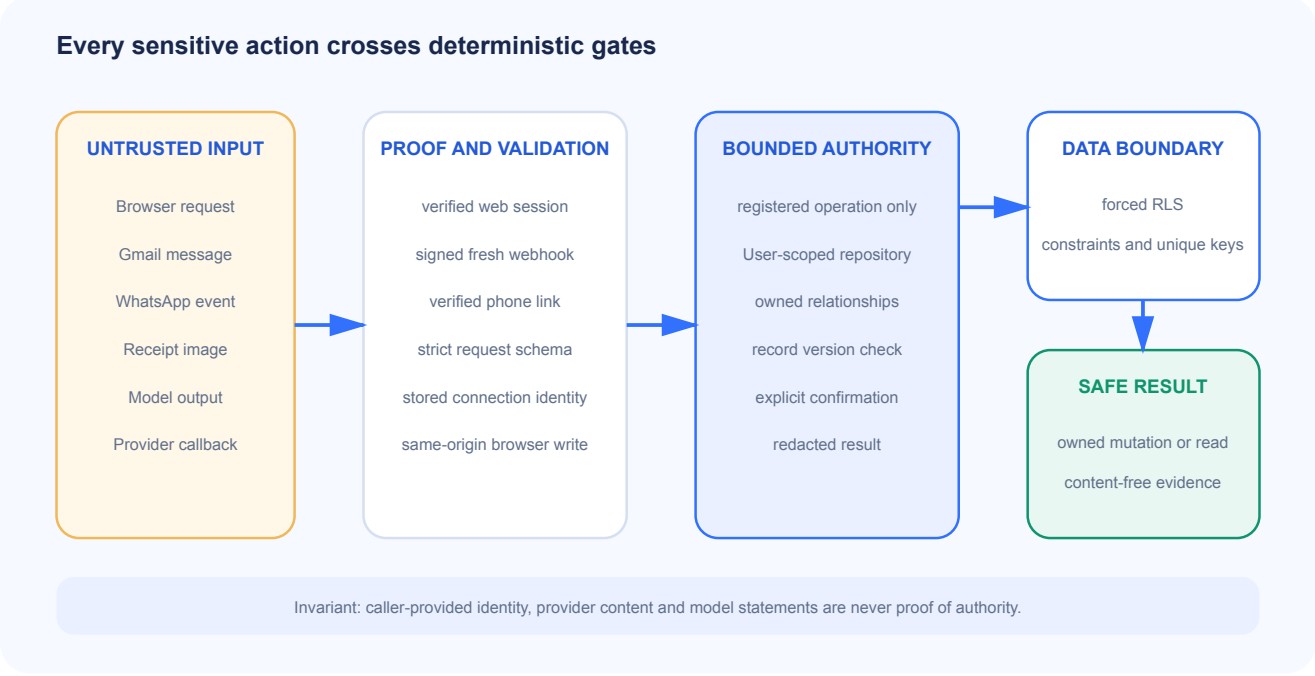
- Deterministic rules run before models so a model cannot invent a financial amount, mapping or forecast candidate.
- Confidence thresholds express product policy, not calibrated real-world accuracy.
- Forecast matching uses normalized merchant names; alternate names can split one merchant into separate histories.
- New Users may have insufficient history for specific predictions; the interface labels limited-history projections.
- Reminder provider acceptance does not prove delivery or reading.
- Recurring bill creation and voice transcription remain Planned.

4 SECURITY DESIGN

Security design

4.1 Trust model

Monara treats browser input, emails, WhatsApp content, receipt images, model output and provider events as untrusted. Each request must cross identity proof, input validation and ownership checks before a bounded operation can execute.



4.2 Identity and authorization

Supabase Google Auth establishes the web User. Gmail authorization is a separate connection and does not reuse application sign-in. A WhatsApp number is keyed, encrypted and usable only after explicit verification by a signed-in User.

Browser writes require a valid session and same-origin request. Agent operations require a server credential plus verified User context. The credential identifies the trusted runtime but never replaces resource ownership checks. Provider connections and background jobs derive ownership from stored records, not caller-supplied identifiers.

4.3 Database ownership and integrity

Every product table enables and forces row-level security. Public callers receive no financial access. Authenticated users can read only selected owned views; sensitive Gmail processing, WhatsApp links, delivery state, confirmation state, Chat history, deletion jobs and forecasts remain backend-only.

Composite relationships bind records to the same User. Knowing another record UUID cannot connect it to an owned transaction, Budget, Category or forecast. Service-level filters, optimistic versions and validation remain mandatory because privileged database access bypasses RLS.

Financial amounts are positive whole IDR values. Category kinds are immutable. Expense Budgets cannot use income or exclusion Categories. Unique constraints protect provider message replay, extracted items, threshold reminders and stored forecast replacement.

4.4 Provider and webhook boundaries

WhatsApp events require the Kapso signature. Gmail trigger events require the Composio signature and freshness check. Missing verification secrets fail closed. Provider response bodies and raw failure messages are not copied into User-facing errors or logs.

Gmail OAuth tokens remain with Composio. Monara stores provider identifiers, bounded extraction evidence and processing state, but not raw Gmail message bodies. Receipt bytes are processed in memory. Brave Search receives only a canonical public merchant name, never personal names, account identifiers, amounts, descriptions or email content.

4.5 Model safety

Model input is delimited and marked as untrusted data. Output must satisfy strict schemas and reference only candidates supplied by deterministic code. The model cannot select identity, bypass an ownership check, directly access the database, execute an unregistered operation or confirm a destructive action.

Prompt controls are treated as defense in depth, not proof of safety. The decisive boundaries are the tool registry, backend allowlist, validated schemas, owned repositories, record versions and database constraints.

4.6 Browser and privacy controls

Authentication, financial and Chat responses are private and non-cacheable. The service worker contains only a public asset allowlist and a generic offline page. Provider keys, model keys, privileged database keys and backend credentials never enter browser bundles.

Private goal images use owned object paths. Upload intents record uncertain outcomes so permanent deletion cannot finish while an object may still require reconciliation. Application logs exclude raw financial records, emails, conversations, authorization headers and tokens.

4.7 Destructive actions and permanent deletion

Reversible Agent writes return a short-lived Undo receipt. Transaction and Budget deletion require a five-minute, resource-bound confirmation. Permanent User deletion is web-only and requires an exact phrase plus acknowledgement. Acceptance immediately blocks the identity and starts durable cleanup.

Cleanup removes owned application rows, conversation state, Gmail triggers and connections, private images and the Auth identity. Each target is leased and retryable. Unfinished external cleanup remains recorded until it can complete; an accepted deletion is never reported as fully complete while a known target remains.

4.8 Security control matrix

Threat	Preventive control	Failure behavior
Cross-User read or write	Verified identity, User-scoped repositories, composite ownership and forced RLS	Request is denied without exposing the target record
Caller identity spoofing	Server-resolved session, stored connection or verified phone link	Caller-supplied IDs do not establish authority
Duplicate provider delivery	Stable provider keys and unique constraints	Retry resolves to the existing logical item
Prompt injection	Untrusted-data framing, strict schemas, registered tools and backend allowlist	Unsupported action fails before financial mutation
Destructive model action	Confirmation stored and executed outside the model	Expired, stale, reused or mismatched confirmation is rejected
Private offline exposure	Public-only cache allowlist and private no-store responses	Offline mode shows a generic reconnect state
Leaked provider failure data	Redacted errors and content-free operational metadata	User receives a bounded error without provider body or secret
Partial account deletion	Immediate identity block plus durable target inventory and retries	Access remains blocked while incomplete cleanup is visible to operators

4.9 Residual risks

- Distributed abuse control depends on deployment-wide counters; a per-instance limit alone is insufficient under horizontal scaling.
- A lost Agent response can leave the User unsure whether a write completed. Chat save-only recovery does not replay the financial action.
- Provider retention, backup erasure and message delivery are governed partly by external systems.
- Model confidence is not a measured probability, and current synthetic evaluation does not establish real-inbox accuracy.
- A crash before reminder delivery reservation can leave an in-app reminder without an outbound row.
- Generalized User timezones, real-device installation behavior and large-population schedule completion need continued acceptance testing.

5 VERIFICATION AND READINESS

Verification and readiness

5.1 Reproducible command gates

Run the following from the repository root on Node 24. The database replay command resets a disposable local stack and must not be pointed at production.

```
corepack pnpm check
corepack pnpm build
corepack pnpm db:check --reset
corepack pnpm db:types --check
```

Expected results:

- formatting, lint, type checks, dependency consistency, unit and integration tests, React Doctor, documentation checks and migration validation pass;
- the Next.js application and Cloudflare Worker build for their target runtimes;
- migrations replay from zero, SQL lint passes, database security tests pass and generated types match the schema.

5.2 Browser acceptance

Use a 390 x 844 mobile viewport and a desktop viewport. Record HTTP status, redirect behavior, manifest availability, horizontal overflow, console errors and page errors. Verify:

- signed-out visitors cannot reach private screens or financial APIs;
- mobile navigation, History filters, review actions, Budgets, Goals and Chat remain usable by touch and keyboard;
- financial and authentication responses remain private and non-cacheable;
- offline mode shows only the generic reconnect state;
- stale or uncertain writes show recovery without substituting sample data.

5.3 Controlled end-to-end journeys

Journey	Pass condition
Identity and isolation	Controlled Users A and B can read and mutate only their own records; sign-out closes financial APIs
Gmail to ledger	Authorized institution mail becomes typed pending or confirmed records; replay creates no duplicate
Review to outputs	Confirm, edit, exclude and delete update only the intended totals; exclusions remain neutral
Agent authority	Web Chat and linked WhatsApp can invoke only registered owned operations; unsigned or replayed input fails closed
Forecast and reminders	Derived values use confirmed records, refresh idempotently and never become financial facts
Permanent deletion	Access is blocked immediately and all known cleanup targets complete or remain explicitly pending

Use synthetic identities and records for production acceptance. Evidence should retain the release identifier, timestamp, observed result and protected boundary, but never cookies, tokens, raw emails, message bodies, phone numbers or financial records.

5.4 Capability status

Area	Status	Acceptance note
Authenticated ledger, review, Budgets, Reports, Goals and insights	Implemented	Requires controlled User isolation and browser acceptance on the deployed release
Gmail authorization, extraction, categorization and deduplication	Implemented	Requires an authorized test mailbox journey and redacted provider evidence
Web Agent, permission tiers, confirmation and Undo	Implemented	Requires uncertain-response and stale-version acceptance
Verified WhatsApp Chat and receipt photos	Implemented	Requires a linked test number and signed provider delivery evidence
Spending forecasts and nightly refresh	Implemented	Deterministic behavior is testable; real-spending accuracy remains unmeasured
In-app Budget reminders	Implemented	Threshold and deduplication behavior are testable
WhatsApp Budget reminder delivery	Implemented with provider dependency	Template approval and valid-recipient delivery require provider acceptance
Voice transcription and bill-due generation	Planned	Not included in implemented claims